LMS Feature Spec: AI Build Document

Stack: React, Node.js (Express), Supabase (Postgres, Auth, Storage, Realtime)

---

0. How to Use This Document (Instructions to the AI Builder)

You are a senior full-stack engineer building a Learning Management System (LMS) from this spec. Follow these rules:

1. **Build in the phase order in Section 12.** Finish and verify each phase before starting the next.
2. **Do not invent features** outside this document. If something is ambiguous, pick the simplest reasonable option, list it under "Assumptions" in the README, and continue.
3. **Write complete, runnable code.** No placeholders, no "TODO", no pseudo-code. Every file you mention must be fully written.
4. **Follow the folder structure, naming conventions and API contracts** defined here exactly, so the frontend and backend stay in sync.
5. **Security first.** Enforce permissions in three places: Supabase Row Level Security (RLS), Node middleware, and React route guards. Never trust the client.
6. **After each phase**, output: files created/changed, how to run it, and a short manual test checklist.
7. Validate every request body with **Zod**. Return errors in the standard format in Section 8.

---

1. Product Overview

A web-based classroom platform where **teachers** publish course content and assessments, **students** consume content and submit work, and **admins** manage users, courses and complaints.

Goals: simple, fast, mobile-friendly, secure. **Non-goals (v1):** payments, video conferencing, mobile native apps, AI features, multi-school tenancy, email notifications beyond password reset.

1.1 Roles

Role	Capabilities
admin	Create/manage users and courses, assign students (and teachers) to courses, view and resolve all complaints, view everything
teacher	Manage own courses' resources, assignments, quizzes, announcements, grading; chat; see which students viewed resources
student	View enrolled courses, view resources (auto-marked seen), submit assignments, take quizzes, read announcements, see own grades, chat, file complaints

Role is stored in `profiles.role` and looked up by the API on each request (or mirrored into the JWT via a custom claim).

---

2. Tech Stack and Tooling

Layer	Choice
Frontend	React 18 + Vite, TypeScript, React Router v6, TanStack Query, React Hook Form + Zod, Tailwind CSS, shadcn/ui, Supabase JS client (auth, realtime and file upload only)
Backend	Node 20, Express, TypeScript, Zod, `@supabase/supabase-js` (service role key, server only), Helmet, CORS, express-rate-limit, Pino logger
Database	Supabase Postgres with RLS enabled on every table
Auth	Supabase Auth (email + password)
Files	Supabase Storage (private buckets, signed URLs)
Realtime	Supabase Realtime (classroom chat)
Testing	Vitest + Supertest (API), React Testing Library (key components)

Layer	Choice
Deployment	Frontend on Vercel/Netlify, API on Render/Railway, Supabase hosted

2.1 Environment Variables

Frontend `.env`: `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `VITE_API_URL`

Backend `.env`: `PORT`, `SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`, `SUPABASE_JWT_SECRET`, `CLIENT_URL` (for CORS and the password reset redirect)

Never expose the service role key to the frontend.

---

3. Architecture

```
React SPA --(JWT in Authorization header)--> Node/Express API --> Supabase Postgres + Storage
    |                                                                    ^
    +---- Supabase Auth (login, reset) and Realtime (chat) -------------+
```

- The React app signs in directly with Supabase Auth and receives a JWT.
- Every call to the Node API sends `Authorization: Bearer <jwt>`.
- Node middleware verifies the JWT, loads the profile (id, role), and attaches `req.user`.
- Business logic lives in Node. RLS is the safety net underneath.
- Chat uses Supabase Realtime subscriptions directly from React (RLS protects the data).

3.1 Folder Structure

```
/lms
  /client
    /src
      /api            # fetch wrappers per feature
      /components     # shared UI (Button, Modal, DataTable, FileUploader)
      /features
        /auth /courses /resources /assignments /quizzes
        /announcements /grading /chat /complaints /users
      /hooks /lib /routes /types
      main.tsx App.tsx
  /server
    /src
      /config         # env, supabase client
```

```
      /middleware     # auth.ts, requireRole.ts, validate.ts, errorHandler.ts
      /modules
        /auth /users /courses /resources /assignments /quizzes
        /announcements /grades /chat /complaints
        # each module: routes.ts, controller.ts, service.ts, schema.ts
      /utils
      app.ts server.ts
  /supabase
    /migrations       # numbered .sql files
    seed.sql
  README.md
```

---

4. Database Schema (Supabase Postgres)

Create as numbered migration files. All primary keys are `uuid default gen_random_uuid()`. All tables have `created_at timestamptz default now()`.

```
create type user_role as enum ('admin','teacher','student');
create type complaint_status as enum
('open','in_review','resolved','rejected');

-- 1:1 with auth.users, created by a trigger on signup
create table profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  full_name text not null,
  email text not null unique,
  role user_role not null default 'student',
  avatar_url text,
  is_active boolean not null default true,
  created_at timestamptz default now()
);

create table courses (
  id uuid primary key default gen_random_uuid(),
  code text not null unique,
  title text not null,
  description text,
  teacher_id uuid not null references profiles(id),
  is_archived boolean default false,
  created_at timestamptz default now()
);

create table enrollments (
  id uuid primary key default gen_random_uuid(),
```

```
  course_id uuid not null references courses(id) on delete cascade,
  student_id uuid not null references profiles(id) on delete cascade,
  assigned_by uuid references profiles(id),
  created_at timestamptz default now(),
  unique (course_id, student_id)
);

create table resources (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses(id) on delete cascade,
  uploaded_by uuid not null references profiles(id),
  title text not null,
  description text,
  file_path text,          -- path in 'resources' bucket
  file_name text,
  mime_type text,
  file_size bigint,
  external_url text,       -- optional link instead of a file
  created_at timestamptz default now(),
  check (file_path is not null or external_url is not null)
);

create table resource_views (   -- the "seen" marker
  id uuid primary key default gen_random_uuid(),
  resource_id uuid not null references resources(id) on delete cascade,
  student_id uuid not null references profiles(id) on delete cascade,
  first_seen_at timestamptz default now(),
  unique (resource_id, student_id)
);

create table assignments (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses(id) on delete cascade,
  created_by uuid not null references profiles(id),
  title text not null,
  instructions text,
  attachment_path text,
  max_points numeric(6,2) not null default 100,
  due_at timestamptz not null,
  allow_late boolean default false,
  created_at timestamptz default now()
);

create table submissions (
  id uuid primary key default gen_random_uuid(),
  assignment_id uuid not null references assignments(id) on delete cascade,
```

```
  student_id uuid not null references profiles(id) on delete cascade,
  text_answer text,
  file_path text,
  file_name text,
  submitted_at timestamptz default now(),
  is_late boolean default false,
  points numeric(6,2),
  feedback text,
  graded_by uuid references profiles(id),
  graded_at timestamptz,
  unique (assignment_id, student_id)   -- resubmission overwrites until
graded
);

create table quizzes (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses(id) on delete cascade,
  created_by uuid not null references profiles(id),
  title text not null,
  description text,
  time_limit_minutes int,            -- null = untimed
  max_attempts int not null default 1,
  opens_at timestamptz,
  closes_at timestamptz,
  is_published boolean default false,
  show_answers_after_close boolean default false,
  created_at timestamptz default now()
);

create table quiz_questions (
  id uuid primary key default gen_random_uuid(),
  quiz_id uuid not null references quizzes(id) on delete cascade,
  position int not null,
  type text not null check (type in
('mcq_single','mcq_multi','true_false','short_text')),
  prompt text not null,
  options jsonb,                     -- [{"id":"a","text":"..."}]
  correct_answer jsonb not null,     -- ["a"] or ["true"] or ["accepted
text"]
  points numeric(5,2) not null default 1
);

create table quiz_attempts (
  id uuid primary key default gen_random_uuid(),
  quiz_id uuid not null references quizzes(id) on delete cascade,
  student_id uuid not null references profiles(id) on delete cascade,
```

```
  started_at timestamptz default now(),
  submitted_at timestamptz,
  answers jsonb default '{}',        -- {question_id: answer}
  score numeric(6,2),
  max_score numeric(6,2)
);

create table announcements (
  id uuid primary key default gen_random_uuid(),
  course_id uuid references courses(id) on delete cascade, -- null = global
(admin only)
  author_id uuid not null references profiles(id),
  title text not null,
  body text not null,
  is_pinned boolean default false,
  created_at timestamptz default now()
);

create table announcement_reads (
  announcement_id uuid references announcements(id) on delete cascade,
  user_id uuid references profiles(id) on delete cascade,
  read_at timestamptz default now(),
  primary key (announcement_id, user_id)
);

create table grade_items (          -- manual grade columns (midterm,
participation...)
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses(id) on delete cascade,
  title text not null,
  max_points numeric(6,2) not null,
  weight numeric(5,2) default 0,
  created_at timestamptz default now()
);

create table manual_grades (
  id uuid primary key default gen_random_uuid(),
  grade_item_id uuid not null references grade_items(id) on delete cascade,
  student_id uuid not null references profiles(id) on delete cascade,
  points numeric(6,2),
  feedback text,
  unique (grade_item_id, student_id)
);

create table chat_messages (
  id uuid primary key default gen_random_uuid(),
```

```
  course_id uuid not null references courses(id) on delete cascade,
  sender_id uuid not null references profiles(id),
  body text not null check (char_length(body) between 1 and 2000),
  is_deleted boolean default false,
  created_at timestamptz default now()
);

create table complaints (
  id uuid primary key default gen_random_uuid(),
  submitted_by uuid not null references profiles(id),
  course_id uuid references courses(id),
  category text not null check (category in
('academic','technical','teacher','student','other')),
  subject text not null,
  description text not null,
  attachment_path text,
  status complaint_status not null default 'open',
  admin_response text,
  resolved_by uuid references profiles(id),
  resolved_at timestamptz,
  created_at timestamptz default now()
);
```

4.1 Indexes

Index all foreign keys, plus: `enrollments(student_id)`, `chat_messages(course_id, created_at desc)`, `submissions(assignment_id)`, `quiz_attempts(quiz_id, student_id)`.

4.2 Storage Buckets (all private)

`resources`, `assignment-files`, `submissions`, `complaint-attachments`, `avatars`. Path convention: `{course_id}/{uuid}-{original_name}`. Serve files only through short-lived signed URLs (60 minutes) generated by the API.

4.3 Row Level Security (summary)

Enable RLS on every table. Core policies:

- **profiles:** user reads own row; teachers read students in their courses; admin reads/writes all.
- **courses:** admin all; teacher reads/writes where `teacher_id = auth.uid()`; student reads courses they are enrolled in.
- **enrollments:** admin all; teacher reads for own courses; student reads own.

- **resources / assignments / quizzes / announcements / grade_items:** teacher of the course can write; enrolled students can read (quizzes only when `is_published`).
- **resource_views:** student inserts/reads own rows; teacher of the course reads all for their resources.
- **submissions:** student reads/writes own (write blocked after grading); teacher reads/updates for own course.
- **quiz_attempts:** student reads/writes own; teacher reads for own course. Students must never read `quiz_questions.correct_answer` while a quiz is open: expose questions to students through the API, which strips it, and give students no direct table access to that column.
- **chat_messages:** course teacher and enrolled students can select and insert; sender can soft-delete own; teacher can soft-delete any in their course.
- **complaints:** submitter reads own; admin reads/updates all.

Provide a SQL helper functions file: `is_admin()`, `is_course_teacher(course_id)`, `is_enrolled(course_id)`.

---

5. Authentication and Authorization (Cross-cutting)

- A trigger on `auth.users` insert creates a `profiles` row (default role `student`; admin sets the role).
- Node `auth` middleware: read bearer token, verify with Supabase, load profile, reject if `is_active = false`.
- `requireRole('admin' | 'teacher' | 'student')` middleware for route protection.
- `requireCourseAccess(courseId)` helper: teacher owns the course, or student is enrolled, or admin.
- React: `AuthProvider` context and a `ProtectedRoute` with an optional `roles` prop; redirect to `/login` when unauthenticated and to the role dashboard when unauthorized.

---

6. Feature Specifications

Each feature lists: user stories, requirements, API endpoints, UI screens, and acceptance criteria.

Feature 1: Login

User stories

- As a user I can log in with email and password.
- As a user I stay logged in after refresh and can log out.
- As an admin I create accounts (no public signup in v1).

Requirements

- Supabase Auth `signInWithPassword` from React. Persist the session (Supabase default).
- After login, call `GET /api/me` and route by role: `/admin`, `/teacher`, `/student`.
- Show clear errors for wrong credentials and deactivated accounts, without revealing which field was wrong.
- Rate limit auth-related API endpoints. Disable the submit button while loading.
- Admin user management: `POST /api/users` (creates the auth user via service role and sets role), `GET /api/users`, `PATCH /api/users/:id` (role, name, active), `DELETE /api/users/:id` (soft deactivate).

UI: Login page (email, password, show/hide password, "Forgot password?" link), role-based layout with sidebar and top bar, profile menu with logout. Admin: Users table with search, role filter, and create/edit modal.

Acceptance criteria

- Valid credentials land on the correct dashboard; invalid ones show an error.
- Visiting a protected URL while logged out redirects to login, then back after login.
- A student cannot open `/teacher` or `/admin` routes, and API calls return 403.

Feature 2: Resource Upload (Teacher) and Seen Marking (Student)

User stories

- As a teacher I upload files (PDF, DOCX, PPTX, images, video up to 100 MB) or add links to a course.
- As a student I see resources of my enrolled courses and open or download them.
- As a student, a resource is marked **Seen** when I open it, and I can tell which ones I have not seen yet.
- As a teacher I see, per resource, which students have and have not seen it, plus a percentage.

Requirements

- Upload flow: client requests a signed upload URL from `POST /api/courses/:courseId/resources/upload-url`, uploads directly to Storage, then calls `POST /api/courses/:courseId/resources` to save metadata. Validate mime type and size server-side.

- Opening a resource calls `POST /api/resources/:id/view` (idempotent, upsert on `(resource_id, student_id)`), then returns a signed URL.
- Students see a "New" badge on unseen resources and a "Seen" check on viewed ones.
- Teacher can edit title/description and delete (delete also removes the storage object).

API

Method	Path	Role
GET	`/api/courses/:courseId/resources`	teacher, enrolled student, admin; includes `seen` boolean for students
POST	`/api/courses/:courseId/resources/upload-url`	teacher
POST	`/api/courses/:courseId/resources`	teacher
PATCH / DELETE	`/api/resources/:id`	teacher (owner)
POST	`/api/resources/:id/view`	student
GET	`/api/resources/:id/views`	teacher: `{seen:[...], notSeen:[...], percent}`

UI: Course > Resources tab: list with icon by type, size, date, Seen/New badge. Teacher: "Add resource" modal with drag-and-drop uploader and progress bar; a "Seen by 18/25" chip that opens a drawer listing students.

Acceptance criteria

- A student not enrolled in the course cannot list or open its resources (403).
- Opening a resource twice creates one `resource_views` row.
- The teacher drawer shows accurate seen/not-seen lists.

Feature 3: Course Assignment to Students

User stories

- As an admin I create courses, assign a teacher, and enroll students (single or bulk).
- As a teacher I can view my roster and enroll students into my courses.
- As a student I see "My Courses".

Requirements

- Course CRUD (admin). `courses.teacher_id` is the owner.
- Enrollment: `POST /api/courses/:courseId/enrollments` with `{studentIds: []}`; ignore duplicates gracefully and report them in the response. Optional stretch: CSV bulk upload of student emails.
- Unenroll with `DELETE /api/courses/:courseId/enrollments/:studentId` (keeps historical grades).
- Course card shows title, code, teacher, student count, and unread announcements badge.

API: `GET/POST /api/courses`, `GET/PATCH/DELETE /api/courses/:id`, `GET /api/courses/:id/students`, enrollment endpoints above, `GET /api/me/courses`.

UI: Admin: Courses table, create/edit form, and a "Manage students" dialog with a searchable multi-select. Student: course card grid. The course page uses tabs: Overview, Resources, Assignments, Quizzes, Announcements, Grades, Chat.

Acceptance criteria

- A newly enrolled student immediately sees the course.
- Duplicate enrollment does not fail the whole request.
- Archived courses are read-only.

Feature 4: Assignments

User stories

- As a teacher I create an assignment with instructions, optional attachment, due date and max points.
- As a student I submit text and/or a file before the deadline and can resubmit until it is graded.
- As a teacher I see all submissions, who is missing, and who is late.

Requirements

- Status per student: `not_submitted`, `submitted`, `late`, `graded`.
- `is_late` is computed server-side by comparing `now()` to `due_at`. If `allow_late = false`, reject submissions after the deadline.
- File types: pdf, doc/docx, zip, images, txt; max 25 MB.
- Grading of submissions is done in Feature 7 but is reachable from this screen.

API

Method	Path	Role
GET / POST	`/api/courses/:courseId/assignments`	list: all in course; create: teacher
GET / PATCH / DELETE	`/api/assignments/:id`	write: teacher
PUT	`/api/assignments/:id/submission`	student (create or replace)
GET	`/api/assignments/:id/submissions`	teacher
GET	`/api/assignments/:id/submission/me`	student

UI: Student: assignment list with due date and status chip; detail page with instructions, upload box, submit button, countdown, and submitted file preview. Teacher: create form, submissions table with filters (all, missing, late, graded) and a link to grade.

Acceptance criteria

- Late submission is flagged or blocked according to `allow_late`.
- Resubmission replaces the previous file and updates `submitted_at`; blocked once graded.
- A student cannot view other students' submissions.

Feature 5: Quizzes

User stories

- As a teacher I build a quiz with multiple question types, a time limit, attempt limit and an open/close window.
- As a student I take a published quiz within its window, with a timer, and see my score.

Question types: `mcq_single`, `mcq_multi`, `true_false`, `short_text` (case-insensitive match against accepted answers).

Requirements

- Quiz builder: add, reorder, edit, delete questions; set points; publish/unpublish. Questions cannot be edited after any attempt exists (warn or offer to clone).
- Starting a quiz: `POST /api/quizzes/:id/attempts` creates an attempt, enforces `max_attempts` and the open window, and returns questions **without** `correct_answer`.
- Autosave answers: `PATCH /api/attempts/:id` every few seconds.

- Submit: `POST /api/attempts/:id/submit`. The server auto-grades and stores `score` and `max_score`. The server enforces the time limit (auto-submit or reject once `started_at + limit` has passed).
- Results: show the score; show correct answers only if `show_answers_after_close` is set and the quiz is closed.
- Quiz scores flow into the gradebook (Feature 7), using the highest attempt by default.

API: `GET/POST /api/courses/:courseId/quizzes`, `GET/PATCH/DELETE /api/quizzes/:id`, `PUT /api/quizzes/:id/questions` (replace the full question list), attempt endpoints above, `GET /api/quizzes/:id/results` (teacher: all attempts, average, per-question stats).

UI: Teacher: quiz builder with live preview, and a results page with score distribution. Student: quiz list with status (upcoming, open, closed, completed), visible countdown timer, and a confirmation modal before submit.

Acceptance criteria

- Correct answers never appear in any student-facing response while the quiz is open.
- Attempt limit and time limit are enforced server-side.
- Auto-grading matches the answer key for every question type (no partial credit in v1).

Feature 6: Announcements

User stories

- As a teacher I post announcements to a course and can pin important ones.
- As an admin I post global announcements for everyone.
- As a student I see announcements and know which are unread.

Requirements

- Fields: title, body (plain text with basic markdown), pinned flag, course or global.
- Track unread state using the `announcement_reads` table and show an unread badge on the course card and sidebar.
- Sorted pinned first, then newest.
- Edit and delete by author or admin.

API: `GET/POST /api/courses/:courseId/announcements`, `GET /api/announcements/global`, `POST /api/announcements` (admin, global), `PATCH/DELETE /api/announcements/:id`, `POST /api/announcements/:id/read`.

UI: Feed layout with pin icon, author, and relative time. Compose modal for teacher/admin. Dashboard widget showing the latest 5 across courses.

Acceptance criteria

- Students only see announcements for enrolled courses plus global ones.
- The unread badge clears after the announcement is opened.

Feature 7: Grading

User stories

- As a teacher I grade assignment submissions with points and feedback.
- As a teacher I create manual grade columns (for example midterm, participation).
- As a student I see my grades and feedback per course and my overall percentage.

Requirements

- Grade sources: assignments (`submissions.points`), quizzes (best attempt `score`), and manual grades.
- Validate `0 <= points <= max_points` .
- Teacher gradebook: students as rows, graded items as columns, inline editing for manual grades, export to CSV.
- Student view: list of items with points, max, percent, feedback, and a course total. Overall percentage = sum of earned points / sum of max points for graded items (or a weighted average if weights are set). Document the formula in the UI.
- Grading a submission sets `graded_by` and `graded_at` and locks resubmission. A teacher can re-open a submission by clearing its grade.

API: `PATCH /api/submissions/:id/grade` `{points, feedback}` , `GET /api/courses/:courseId/gradebook` (teacher), `GET /api/courses/:courseId/grades/me` (student), `POST/PATCH/DELETE /api/courses/:courseId/grade-items` , `PUT /api/grade-items/:id/grades` (bulk save of manual grades), `GET /api/courses/:courseId/gradebook.csv` .

UI: Teacher grading screen with submission preview on the left and points/feedback form on the right, plus next/previous student buttons. Gradebook grid. Student grades page with a progress bar.

Acceptance criteria

- Points above max are rejected with a clear message.
- A student sees only their own grades, and only after they are graded.
- Totals in teacher and student views match.

Feature 8: Classroom Chat

User stories

- As a student or teacher I chat in a real-time room per course.
- As a teacher I can delete inappropriate messages.

Requirements

- One chat room per course, accessible only to the course teacher, enrolled students and admin.
- Use Supabase Realtime (`postgres_changes` on `chat_messages` , filtered by `course_id`) for live updates.
- Send via `POST /api/courses/:courseId/messages` (length 1 to 2000, rate limit 20 messages per minute per user). History via `GET /api/courses/:courseId/messages?before=<timestamp>&limit=50` with infinite scroll upward.
- Soft delete: `DELETE /api/messages/:id` (sender, course teacher or admin) sets `is_deleted = true` and shows "message deleted".
- Show sender name, a Teacher role badge, time, grouped by day. Auto-scroll to the newest message unless the user scrolled up, in which case show a "New messages" pill.
- Escape all content to prevent XSS; no HTML rendering. Optional: online indicator via Realtime Presence.

Acceptance criteria

- Two browsers in the same course see messages instantly.
- A user outside the course cannot read or subscribe to the room (RLS verified).
- Deleted messages disappear for everyone in real time.

Feature 9: Complaints

User stories

- As a student or teacher I submit a complaint with category, subject, description, optional course and attachment.
- As a user I track the status and read the admin response.
- As an admin I review, change status and respond.

Requirements

- Status flow: `open` > `in_review` > `resolved` or `rejected` . Setting resolved or rejected requires an `admin_response` .
- Users see only their own complaints; admin sees all, with filters (status, category, course, date) and search.
- Attachment: image or PDF up to 5 MB in `complaint-attachments` .
- Complaints cannot be edited after submission in v1.

- Admin dashboard shows a counter of open complaints.

API: `POST /api/complaints`, `GET /api/complaints/mine`, `GET /api/complaints` (admin, with query filters and pagination), `GET /api/complaints/:id`, `PATCH /api/complaints/:id` (admin: `{status, admin_response}`).

UI: "New complaint" form; "My complaints" list with status chips; admin table with a detail drawer and respond/resolve actions.

Acceptance criteria

- Non-admins cannot read others' complaints.
- Status changes record `resolved_by` and `resolved_at`.

Feature 10: Reset Password

User stories

- As a user who forgot my password I request a reset link by email and set a new password.
- As a logged-in user I can change my password from settings.

Requirements

- Forgot password page: the email field calls `supabase.auth.resetPasswordForEmail(email, { redirectTo: CLIENT_URL + '/reset-password' })`. Always show the same generic success message whether or not the email exists.
- `/reset-password` page: detect the recovery session, show new password and confirm fields, call `supabase.auth.updateUser({ password })`, then redirect to login.
- Password rules: minimum 8 characters, at least one letter and one number, with a strength meter.
- Change password in profile: require the current password (re-authenticate), then update.
- Document in the README how to configure Supabase email templates and the redirect URL allow-list.
- Handle an expired or invalid link with a clear message and a button to request a new one.

Acceptance criteria

- The reset email arrives and the link opens the reset page.
- After reset, the old password fails and the new one works.
- Reusing an expired link shows an error, not a blank page.

---

7. Navigation Map

Role	Pages
Public	`/login`, `/forgot-password`, `/reset-password`
Student	`/student` dashboard, `/courses`, `/courses/:id/*` (tabs), `/grades`, `/complaints`, `/profile`
Teacher	`/teacher` dashboard, `/courses`, `/courses/:id/*` (tabs incl. gradebook, quiz builder), `/complaints`, `/profile`
Admin	`/admin` dashboard, `/users`, `/courses`, `/complaints`, `/announcements`, `/profile`

---

8. API Conventions

- Base path `/api`, JSON only, snake_case in the DB and camelCase in API responses.
- Success: `{ "data": ..., "meta": { "page": 1, "pageSize": 20, "total": 100 } }` (meta only for lists).
- Error: `{ "error": { "code": "VALIDATION_ERROR", "message": "Human readable", "details": [] } }`.
- Status codes: 200, 201, 204, 400 (validation), 401 (no or invalid token), 403 (wrong role or not enrolled), 404, 409 (conflict), 429, 500.
- Paginate all list endpoints with `page` and `pageSize` (default 20, max 100).
- Every route passes through: `auth` > `requireRole` > `validate(zodSchema)` > controller > service. Controllers stay thin.

---

9. Frontend Conventions

- TypeScript strict mode. Shared types live in `/types` and mirror API shapes.
- Data fetching only through TanStack Query hooks (`useCourses`, `useResources`, and so on). Invalidate queries after mutations.
- Every screen implements **loading, empty, error and success** states.
- Forms: React Hook Form + Zod resolvers, inline field errors, submit disabled while pending, toast on success or failure.
- Responsive from 360px up. Accessible: labels, focus states, keyboard-navigable modals, sufficient contrast.

- Confirm dialogs for destructive actions (delete, unenroll).

---

10. Non-Functional Requirements

- **Security:** Helmet, strict CORS to `CLIENT_URL`, rate limiting, input validation everywhere, signed URLs only, no secrets in the client bundle, sanitized user text, parameterized queries only.
- **Performance:** paginate lists, index foreign keys, avoid N+1 queries, lazy-load route bundles.
- **Reliability:** centralized error handler, structured logs (never log passwords or tokens), `GET /api/health` endpoint.
- **Data integrity:** DB constraints and transactions for multi-step writes (for example submitting a quiz and computing the score).
- **Timezones:** store UTC, display in the user's local time.

---

11. Seed Data and Testing

The seed script must create: 1 admin, 2 teachers, 10 students, 3 courses with enrollments, sample resources, 2 assignments, 1 published quiz with 5 mixed questions, 2 announcements, and a few chat messages. Print the seeded credentials in the README (dev only).

Minimum tests: auth middleware, role guards, enrollment access check, late submission logic, quiz auto-grading for all four question types, grade bounds validation, and complaint status transitions.

---

12. Build Order (Phases)

Phase	Scope	Done when
1	Repo setup, Supabase project, migrations, RLS helpers, Express skeleton, Vite skeleton, env handling	Both apps run, `/api/health` works
2	**Login** + user management + **Reset password** + role-based layouts	All roles can log in, reset flow works
3	**Courses** and **course assignment** to students	Admin enrolls students, students see courses

Phase	Scope	Done when
4	**Resources** with upload and seen tracking	Upload, view and seen report all work
5	**Announcements**	Feed with unread badges
6	**Assignments** and submissions	Submit, late logic, teacher review
7	**Grading** and gradebook (assignments + manual)	Teacher grades, student sees grades
8	**Quizzes** (and quiz scores in the gradebook)	Full attempt lifecycle works
9	**Classroom chat** (realtime)	Live messaging with RLS verified
10	**Complaints**	Submit, track, resolve
11	Polish: empty states, error handling, accessibility, seed data, tests, README, deployment guide	Checklist in Section 13 passes

---

13. Definition of Done (Final Checklist)

- [ ] All 10 features meet their acceptance criteria
- [ ] RLS enabled on every table and tested with each role
- [ ] No secret keys in the client; `.env.example` files provided
- [ ] All endpoints validated with Zod and returning the standard error shape
- [ ] Every page has loading, empty and error states and works on mobile
- [ ] Seed script runs cleanly on a fresh Supabase project
- [ ] README covers setup, env vars, Supabase dashboard steps (auth redirect URLs, email templates, storage buckets), run scripts and deployment
- [ ] Test suite passes

---

14. Assumptions Made (Change If Wrong)

1. No public signup; admin creates all accounts.
2. One teacher per course (can be extended to many later).
3. "Course assignment" means admin or teacher enrolling students into courses.
4. Chat is one room per course (no private DMs in v1).
5. "Seen" means the student opened the resource at least once.

6. Email delivery is limited to Supabase Auth emails (password reset).

---

15. Starter Prompt to Paste Into Your AI Tool

> You are a senior full-stack engineer. Read the attached LMS Feature Spec in full. Build the project using React (Vite, TypeScript), Node/Express (TypeScript) and Supabase exactly as specified. Start with **Phase 1** only: output the complete file tree, all SQL migrations, and all code for the setup. When done, list how to run it and a test checklist, then wait for me to say "continue" before starting the next phase. Do not skip sections, do not use placeholders, and ask me only if something is truly blocking.